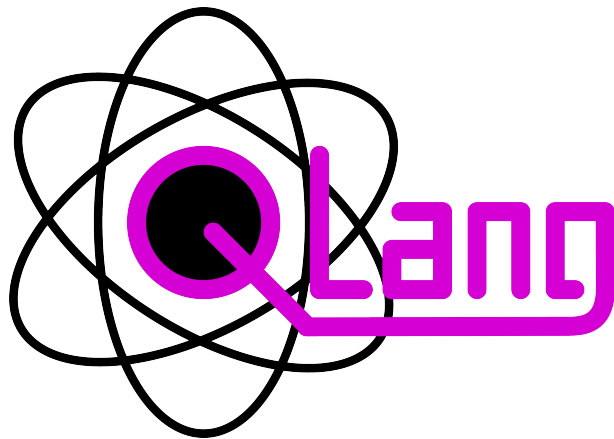




QLang Programming Language

Language Specification & Documentation

Created by:

Kamil Widzyk
Maciej Świątek
Jan Wida

Contents

1	Introduction	2
1.1	Overview	2
1.2	Quick Example	2
1.3	Design Principles	3
1.3.1	Classical and Quantum Code in One Language	3
1.3.2	Structured Error Reporting	3
1.3.3	Modular Structure via Places	3
2	Language Basics	4
2.1	Program Structure	4
2.2	Data Types	4
2.3	Variables	4
2.3.1	Declaration Syntax	5
2.3.2	Arrays	5
2.3.3	Constants	5
2.3.4	The # Size Getter	5
2.4	Operators	6
2.4.1	Arithmetic Operators	6
2.4.2	Comparison and Logical Operators	6
2.4.3	Compound Assignment Operators	6
2.4.4	Increment and Decrement	6
2.4.5	String Formatting Operator	6
2.5	Control Flow	7
2.5.1	Conditional Statements	7
2.5.2	For Loop	9
2.5.3	While Loop	9
2.5.4	Break and Continue	9
2.6	Functions	9
2.6.1	Named Arguments	10
2.6.2	Variadic Parameters	10
2.7	Input and Output	11
2.7.1	Output	11
2.7.2	Input	12
3	Quantum Computing	13

1 Introduction

QLang is a high-level programming language designed for quantum computing. It provides a simple and intuitive syntax for expressing quantum algorithms and operations, making it accessible to both beginners and experienced programmers in the field of quantum computing.

1.1 Overview

A QLang program is made up of top-level statements and **place** declarations. A place is QLang's primary unit of encapsulation, similar to a class or module in classical languages. Each place can define functions and execute statements. The special *global place*, code written outside any place, acts as a default place called **global**.

The language supports four built-in variable types: **state**, **obs**, **num**, and **text**. Quantum-specific operations, gates such as **H**, **X**, **CNOT**, and the **measure** keyword, are first-class constructs in QLang, not library calls. QLang is interpreted. Source files (extension **.ql**) are parsed by an ANTLR4-generated parser and executed by the QLang interpreter written in Python.

1.2 Quick Example

The following complete QLang program creates a Bell state (a maximally entangled two-qubit state), measures both qubits, and prints the results:

```
place Alice {  
    state a, b;  
  
    H a;           // Put qubit 'a' into superposition  
    CNOT a -> b;   // Entangle 'a' with 'b' - this creates a Bell  
state  
  
    obs ma = measure a;  
    obs mb = measure b;  
  
    println("Measurement A: %d" % ma);  
    println("Measurement B: %d" % mb);  
    // Both will always agree: both 0 or both 1  
}
```

After running, both measurements will always agree, both **0** or both **1**, which is the defining property of a Bell state. This example demonstrates how QLang lets you express a complete quantum protocol in just a few lines.

1.3 Design Principles

1.3.1 Classical and Quantum Code in One Language

QLang does not separate quantum and classical concerns into different APIs or files. A variable of type **obs** (classical integer) and a variable of type **state** (qubit) can appear side-by-side in the same scope. Measurement, when invoked on a **state**, collapses the qubit to a classical value (**obs**) and returns a **bool** indicating the outcome: **false** if the qubit collapsed to $|0\rangle$, **true** if it collapsed to $|1\rangle$.

1.3.2 Structured Error Reporting

When the interpreter encounters an error, whether a syntax mistake, an undefined variable, or an illegal runtime operation, it displays a detailed error box that highlights the exact source location, shows surrounding context lines, and explains the cause.

1.3.3 Modular Structure via Places

Each **place** in a QLang program runs as a logically isolated process with its own variable scope and console output. Places communicate via the **send** and **received** primitives, which allow passing both classical (**obs**) and quantum (**state**) data between places. The **received** instruction blocks execution until the expected message is received, enabling synchronisation between parallel processes. This makes it possible to express distributed quantum protocols such as quantum teleportation directly in the language.

2 Language Basics

This chapter covers the fundamental building blocks of QLang: how programs are structured, the available data types, variables, operators, and control flow constructs.

2.1 Program Structure

A QLang source file (extension `.ql`) consists of a sequence of top-level items. Each item is one of the following:

- A **place** declaration.
- A function declaration (at global place).
- A statement (at global place).

Items are executed in the order they appear. The global place is the implicit default place and is always present. When a named place is declared, its body executes when the program starts.

2.2 Data Types

QLang has four built-in variable types, designed to cover both classical and quantum computation:

Type	Description
state	A quantum state (qubit or qubit register). Represents a quantum system that can be in superposition and can be entangled with other qubits. Collapsed to a classical value via the measure keyword.
obs	An observation, a classical integer. Stores the result of a measured state or any integer value. Supports arithmetic, comparison, and bitwise operations. Bit-width is specified as obs x[N] .
num	A universal numeric type. It dynamically handles both integers and floating-point numbers, automatically adapting its internal representation based on the assigned value and performed operations. Used for all classical numerical computations.
text	A string of characters. Used for textual data, print formatting, and string operations.

2.3 Variables

Variables are declared by specifying a type, a name, an optional bit-width (for **obs** and **state**), and an optional initial value.

2.3.1 Declaration Syntax

```
num pi = 3.14159;           // floating-point number
num count = 10;             // num accepts integer values too
obs result[16];             // 16-bit integer register, default value
0
obs counter[8] = 0;         // 8-bit integer register, initialized to
0
text greeting = "Hello!";   // string
```

2.3.2 Arrays

Any variable type can be declared as an array by appending one or more size specifiers in square brackets. Arrays are zero-indexed. A size can be a fixed integer literal or an expression evaluated at runtime:

```
num matrix[4][4];           // two-dimensional 4x4 array

// Size inferred automatically from the initialiser
num a[?] = [1, 2, 3, 4, 5]; // size is 5, inferred from the list
println(#a);                // 5

// Runtime-computed size
num n = 8;
num dynamic[n];             // size determined at runtime
```

2.3.3 Constants

Variables declared with the **const** keyword cannot be reassigned after their initial value is set. Attempting to modify a constant at runtime produces an error.

```
const obs MAX_SIZE[8] = 100;
const num EPSILON = 0.0001;
const PI = 3.14159;
```

2.3.4 The # Size Getter

The **#** prefix operator returns the declared length of an array:

```
num items[?] = [10, 20, 30, 40, 50];
println(#items);    // 5
```

2.4 Operators

QLang supports the standard set of classical operators as well as quantum-specific operations.

2.4.1 Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulo (remainder)
**	Exponentiation (power)

2.4.2 Comparison and Logical Operators

Operator	Description
==	Equal to
!=	Not equal to
<, >, <=, >=	Relational comparisons
&&	Logical AND
	Logical OR
!	Logical NOT

2.4.3 Compound Assignment Operators

In addition to plain assignment (=), QLang supports compound operators: +=, -=, *=, /=, %=, **=, &=, |=.

2.4.4 Increment and Decrement

```
num i = 0;

i++;    // post-increment
++i;    // pre-increment
i--;    // post-decrement
--i;    // pre-decrement
```

2.4.5 String Formatting Operator

The % operator, when the left operand is a **text** string, performs printf-style interpolation:

```
num x = 42;
num y = 3.14159;
println("The answer is %d" % x);
println("x = %d, y = %.2f" % [x, y]); // list of values

text fmt = "Hello, %s!";
println(fmt % "world");
```

2.5 Control Flow

2.5.1 Conditional Statements

QLang supports standard `if / else if` (or `elif`) / `else` branching:


```

num x = 75;

if (x > 100) {
    println("Large");
} else if (x > 50) {
    println("Medium");
} else {
    println("Small");
}

// if without braces
if (x > 0) println("Positive");

// Ternary operator
text label = x > 50 ? "big" : "small";
println(label);

// For loop with step
for i from 0 to 20 step 5 {
    println(i);    // 0, 5, 10, 15
}

// While loop
num n = 1;
while (n < 128) {
    n *= 2;
}

println(n);    // 128

```

2.5.2 For Loop

The **for** loop iterates over a numeric range with an optional step. The syntax is **for** <var> **from** <start> **to** <end> **step** <step>. When **step** is omitted, the step defaults to 1.

2.5.3 While Loop

The **while** loop repeats its body as long as the condition holds.

2.5.4 Break and Continue

break exits the nearest enclosing loop immediately. **continue** skips the rest of the current iteration and proceeds to the next one.

2.6 Functions

Functions are declared with the **function** keyword. Parameters are typed, can have default values, and may be arrays. A function returns a value with the **return** statement.

```
function add(num a, num b) {  
    return a + b;  
}  
  
function greet(text name = "World") {  
    println("Hello, " + name + "!");  
}  
  
function power(num base, num exp) {  
    return base ** exp;  
}  
  
greet();                                // Hello, World!  
greet("Alice");                         // Hello, Alice!  
println(add(3, 4));                     // 7  
println(power(exp = 3, base = 2));      // 8
```

2.6.1 Named Arguments

Arguments can be passed by name, which allows them to be supplied in any order and makes call sites more self-documenting:

```
function rect(num width, num height = 1) {  
    return width * height;  
}  
  
// Positional arguments  
println(rect(5, 3));           // 15  
  
// Mixed: positional + named  
println(rect(5, height = 3));  // 15  
  
// All named, any order  
println(rect(height = 4, width = 6)); // 24  
  
// Using default value  
println(rect(7));              // 7
```

2.6.2 Variadic Parameters

A function can accept a variable number of arguments using the `...` prefix. Inside the function, these are available as an array:

```

function sum(num ...values) {
    num s = 0;
    for i from 0 to #values {
        s += values[i];
    }
    return s;
}

println(sum(1, 2, 3));           // 6
println(sum(10, 20, 30, 40));   // 100

// Regular parameters can precede the variadic one
function prefixed(num x, num ...rest) {
    println("x=" + x + " count=" + #rest);
}

prefixed(5, 10, 15);           // x=5 count=2

```

2.7 Input and Output

2.7.1 Output

QLang provides three output statements:

- `print(...)` — prints values without a trailing newline.
- `println(...)` — prints values followed by a newline.
- `debug(x)`

Both `print` and `println` accept an optional format specifier (**BIN** or **HEX**) as a second argument:

```
// Basic output
print("no newline");
println(" with newline");
println(); // blank line

// Multiple arguments
println(1, 2, 3); // 1 2 3

// Printf-style formatting
num val = 42;
println("The answer is %d" % val);
println("x = %d, y = %.2f" % [val, 3.14]);
```

2.7.2 Input

The **input** statement reads a value from the user into a variable. An optional constraint limits the accepted range, either with the range notation or the **range()** function:

```
obs age[8];
input(age); // read any value

obs score[8];
input(score, 0..100); // accept only values in range [0, 100]
input(score, range(0, 100)); // equivalent syntax
```

3 Quantum Computing